

ARM64 (AArch64) Hex Patch Reference

for GameGuardian & IL2CPP / Unity modding

Corrected and expanded reference of the most-used ARM64 opcodes for boolean bypass, return-value force, NOP, and short-circuit patches. Based on the Arm Architecture Reference Manual (A64), verified encodings, and common GameGuardian practice.

Primary sources: *Arm A-profile Architecture Reference Manual (DDI0487)* · *Arm A64 Instruction Set Reference (NOP, RET, MOV)* · *Apple Id64 NOP encoding* · *Go golang.org/x/arch/arm64asm tables* · *PlatinMods / GameGuardian community patterns*

1. Why these patches work

AArch64 uses a **fixed 32-bit instruction length**. Every instruction is exactly 4 bytes. In IL2CPP / Unity games, C# methods compile to sequences of these words. Under the AAPCS64 calling convention, integer and pointer results return in **X0** (64-bit) or **W0** (lower 32 bits of the same register). Boolean **true** is typically 1, **false** is 0. Overwriting the start of a function with `MOV X0, #1 + RET` forces an immediate true return.

GameGuardian edits these 32-bit words as hex (often with an h prefix). Prefer editing full dwords rather than scrambling byte order by hand.

2. Corrected core opcode table

Your screenshot is accurate. Cleaned table with dword form and little-endian byte form used in raw memory:

Hex (dword)	Bytes (LE)	ARM64	Meaning
D503201F	1F 20 03 D5	NOP	No operation — skip this instruction
D65F03C0	C0 03 5F D6	RET	Return to caller (via LR)
D2800000	00 00 80 D2	MOV X0, #0	Load 0 into X0 (false / zero)
D2800020	20 00 80 D2	MOV X0, #1	Load 1 into X0 (true)
D2800040	40 00 80 D2	MOV X0, #2	Load 2 into X0
D2800060	60 00 80 D2	MOV X0, #3	Load 3 into X0
52800000	00 00 80 52	MOV W0, #0	32-bit: 0 into W0
52800020	20 00 80 52	MOV W0, #1	32-bit: 1 into W0
52800040	40 00 80 52	MOV W0, #2	32-bit: 2 into W0
2A1F03E0	E0 03 1F 2A	MOV W0, WZR	Zero register → W0 (=0)
AA1F03E0	E0 03 1F AA	MOV X0, XZR	Zero register → X0 (=0)
14000000	00 00 00 14	B #0	Branch to self (loop — crash risk)

Correction: NOP is **D503201F** (bytes 1F 20 03 D5). RET is **D65F03C0**. Avoid garbled/reversed forms of NOP.

3. How MOV immediate is encoded

Small constants use **MOVZ** (Move Wide with Zero), which assemblers show as `MOV`. For 64-bit X registers the base is `D2800000`; the 16-bit immediate sits in bits [20:5]:

```
MOV X0, #N ≈ D2800000 | (N << 5)
```

```
#0 → D2800000 #1 → D2800020 #2 → D2800040 #3 → D2800060
```

```
32-bit W form uses base 52800000 with the same immediate shift.
```

XZR/WZR is register 31 as a hardwired zero source. `MOV X0, XZR` encodes to `AA1F03E0` (alias of `ORR` from the zero register).

4. Most-used 2-instruction combos

Force TRUE (IsBought / IsUnlocked / bool)

```
MOV X0, #1 ; RET  
→ D2800020 D65F03C0  
→ GG: h200080D2 hc0035FD6 or ~A8 MOV X0, #1 + ~A8 RET
```

Force FALSE

```
MOV X0, #0 ; RET  
→ D2800000 D65F03C0  
→ GG: h000080D2 hc0035FD6
```

32-bit W0 variants

```
MOV W0, #1 ; RET → 52800020 D65F03C0 (h20008052 hc0035FD6)  
MOV W0, #0 ; RET → 52800000 D65F03C0 (h00008052 hc0035FD6)
```

NOP one instruction

```
NOP → D503201F (bytes 1F 20 03 D5)  
Use to neutralize a branch/check without changing instruction count.
```

5. X0 vs W0 — which to use?

W0 is the lower 32 bits of X0 (same physical register). Writing W0 zero-extends into X0.

Prefer X0 (#1 / #0) for bool, object pointers, long/ulong, most IL2CPP bool/reference returns.

Prefer W0 when disassembly clearly uses W registers, or for explicit 32-bit int/enum returns.

If a patch crashes, switch width. Also try MOV X0, XZR / MOV W0, WZR instead of immediate #0.

6. Practical GameGuardian usage

In scripts (SecreDevPatch style)

```
SecreDevPatch({
  {0x0, "D2800020h"}, -- MOV X0, #1
  {0x4, "D65F03C0h"}, -- RET
}, OFF_IsBought)

-- Assembly form if GG supports ~A8:
{0x0, "~A8 MOV X0, #1"}, {0x4, "~A8 RET"},
```

Manual edit steps

1. Take method RVA from dump.cs → add to libil2cpp.so CODE_APP base.
2. Open as DWORD / hex view.
3. Replace the first instruction(s) with the MOV+RET pair.
4. Always end the forced path with RET if you overwrote the original epilogue.

7. Common mistakes

Mistake	Symptom	Fix
Wrong NOP hex	Garbled/reversed bytes	Use D503201F / 1F 20 03 D5
Missing RET	Falls into next function code	Always pair with D65F03C0
W0 vs X0 mismatch	Crash or wrong result	Match disassembly; try both
Patch mid-function	Broken stack / registers	Patch at method entry
B #0 (14000000)	Freeze / ANR	Avoid unless intentional
Endian mix-up	Wrong bytes in memory	Prefer full dword form in GG

8. Branches (advanced)

Unconditional B imm uses the 0x14xxxxxx family. Offset is a signed instruction count (imm26), not bytes. B #0 (14000000) jumps to itself → hang. Safer approach: NOP the failing conditional branch, or replace the whole function with MOV+RET.

9. Float returns (brief)

Floats use FP registers (S0/D0), not X0. Simple integer MOV tricks do not apply. For unlock/bool work you almost never need float opcodes.

10. One-page cheat sheet

Goal	Hex dwords	GG h-form
Always TRUE	D2800020 + D65F03C0	h200080D2 hC0035FD6
Always FALSE	D2800000 + D65F03C0	h000080D2 hC0035FD6
TRUE (W0)	52800020 + D65F03C0	h20008052 hC0035FD6
FALSE (W0)	52800000 + D65F03C0	h00008052 hC0035FD6

Goal	Hex dwords	GG h-form
FALSE via XZR	AA1F03E0 + D65F03C0	hE0031FAA hC0035FD6
NOP one insn	D503201F	h1F2003D5
RET only	D65F03C0	hC0035FD6

Remember: Prefer patching the method entry. Confirm the process is ARM64. Keep original bytes for restore. Official encodings live in the Arm Architecture Reference Manual for A-profile (A64).

Educational reference for learning AArch64 encoding and offline reverse engineering.